

AI Web Search v2 — Agent Loop with Tool Use

 Open in Colab

Input: User Query

Architecture: Agent Loop with Tool Use and Multiple LLM Calls

1. Define a `web_search` function as a **tool** the LLM can invoke via the Tavily API.
2. Run an **Agent Loop** (up to `MAX_LOOP_TIMES` LLM calls):
 - The LLM decides when and what to search.
 - Each search result is labeled `[n]`; results accumulate and are deduplicated across iterations.
 - Loop exits when the LLM returns an answer instead of a tool call.
3. Render the output as HTML in this notebook:
 - **Answer Section** — LLM answer with inline citations `[n] (URL)`
 - **Web Search Results Section** — all unique results in discovery order

```
# Install dependencies (run once)
%pip install -q litellm tavily-python markdown
```

```
17.1/17.1 MB 68.3 MB/s eta 0:00:00
278.1/278.1 kB 20.4 MB/s eta 0:00:00
```

```
import os, json, re
import litellm
from tavily import TavilyClient
from IPython.display import display, HTML
import markdown

# --- API Keys -----
TAVILY_API_KEY = "tvly-..." # your Tavily API key
OPENAI_API_KEY = "sk-..." # your OpenAI API key
os.environ["OPENAI_API_KEY"] = OPENAI_API_KEY

# --- Configuration -----
MODEL = "gpt-4o-mini" # change to e.g. "gpt-4o", "claude-3-5-haiku-20241022"
TOP_K = 10 # max web results per search
MAX_LOOP_TIMES = 3 # max LLM calls per request (including final answer)

litellm.set_verbose = False
```

Step 1: Define Web Search as a Tool

`search_web` fetches results from Tavily. `WEB_SEARCH_TOOL` is the function schema exposed to the LLM so it can call this tool when needed.

```
def search_web(query, k=TOP_K):
    """Return top-k Tavily results for query."""
    response = TavilyClient(api_key=TAVILY_API_KEY).search(query, max_results=k)
    return response["results"] # each: {title, url, content, score}

WEB_SEARCH_TOOL = {
    "type": "function",
    "function": {
        "name": "web_search",
        "description": "Search the web for current information on a topic.",
        "parameters": {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "The search query"}
            },
            "required": ["query"]
        }
    }
}
```

✓ Step 2: Agent Loop

The loop initializes the conversation with the user query, then repeatedly calls the LLM:

- If the LLM issues a `web_search` tool call, we execute the search, number the new results globally (`[n]`), deduplicate by URL, and add the results to the message context.
- If the LLM returns a plain text response (no tool call), we exit and return it as the answer.

The loop runs at most `MAX_LOOP_TIMES` times (= max total LLM calls).

```
SYSTEM_PROMPT = (
    "You are a helpful research assistant with access to a web_search tool. "
    "Use it to find information needed to answer the user's query; you may search multiple times with different quer
    "Each search result is labeled [n] with its URL. "
    "Cite supporting text in your answer as [n](URL) using those numbers. "
    "When you have enough information, write a clear, comprehensive markdown answer."
)

def run_agent(query, model=MODEL, max_loops=MAX_LOOP_TIMES):
    """Run the agent loop; return (answer_text, all_results, model_name)."""
    seen_urls = {} # url -> result dict (augmented with "num" key)
    counter = 0 # global discovery counter across all searches
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": query},
    ]

    last_content = ""
    model_name = model
    for loop_num in range(max_loops):
        print(f"\n[Loop {loop_num + 1}] Calling LLM...")
        response = litellm.completion(
            model=model, messages=messages,
            tools=[WEB_SEARCH_TOOL], tool_choice="auto"
        )
        msg = response.choices[0].message
        model_name = response.model # actual model name returned by the API
        last_content = msg.content or ""

        if not msg.tool_calls: # LLM produced a plain answer – exit loop
            print(f"[Loop {loop_num + 1}] LLM returned answer. Exiting loop.")
            return last_content, list(seen_urls.values()), model_name

        # --- Execute tool calls and add results to context ---
        messages.append(msg)
        for tc in msg.tool_calls:
            search_query = json.loads(tc.function.arguments)["query"]
            print(f"[Loop {loop_num + 1}] Tool call: web_search('{search_query}')"
            results = search_web(search_query)
            print(f"[Loop {loop_num + 1}] Got {len(results)} results")

            tool_text = ""
            for r in results:
                if r["url"] not in seen_urls: # deduplicate; first occurrence wins
                    counter += 1
                    r["num"] = counter
                    seen_urls[r["url"]] = r
                num = seen_urls[r["url"]]["num"]
                tool_text += f"[{num}] {r['title']}\nURL: {r['url']}\n{r['content']}\n\n"

            messages.append({
                "role": "tool", "tool_call_id": tc.id,
                "name": "web_search", "content": tool_text
            })

        # Max loops reached – return whatever the LLM last said
        print(f"[Loop {max_loops}] Max loops reached. Returning last LLM response.")
        return last_content, list(seen_urls.values()), model_name
```

✓ Step 3: Format and Display Output

`format_output` assembles the final markdown:

- **Answer** section: the LLM's response (with `[n] (URL)` citations).
- **Web Search Results** section: all unique results in discovery order (`[1]`, `[2]`, ...).

`display_answer` converts markdown to HTML and renders it in the notebook.

```
def format_output(answer_text, all_results, model_name):
    """Combine LLM answer and search results into a single markdown string."""
    # Normalize LLM citations: replace full-width 【 with standard [
    answer_text = answer_text.replace('【', '[')
    # Wrap citation numbers in brackets so [2](URL) renders as [2] instead of 2
    answer_text = re.sub(r'\s*([0-9]+)\s*\(', r'[{}](\s*'.format('['), answer_text)

    # Display results in ascending [n] (discovery) order
    ordered = sorted(all_results, key=lambda r: r["num"])

    results_md = "## Web Search Results\n\n"
    for r in ordered:
        results_md += f"- [{r['num']}] [{r['title']}]({r['url']})\n\n"

    return f"## Answer *(model: {model_name})*\n\n{answer_text}\n\n---\n\n{results_md}"

def display_answer(md_text):
    """Render markdown as HTML in the notebook."""
    html = markdown.markdown(md_text, extensions=["extra"])
    display(HTML(html))
```

✓ Run the Demo

Set your query below and run the cell. The agent will search the web as needed and display a cited answer.

```
query = "How was Claude Code implemented?" # <- change this

answer_text, all_results, model_name = run_agent(query)
output_md = format_output(answer_text, all_results, model_name)
display_answer(output_md)
```

```
[Loop 1] Calling LLM...
[Loop 1] Tool call: web_search('Claude Code implementation details')
[Loop 1] Got 10 results

[Loop 2] Calling LLM...
[Loop 2] Tool call: web_search('Claude Code architecture and features')
[Loop 2] Got 10 results

[Loop 3] Calling LLM...
[Loop 3] LLM returned answer. Exiting loop.
```

Answer (model: gpt-4o-mini-2024-07-18)

Claude Code Implementation Overview

Claude Code is an advanced AI-assisted coding tool developed by Anthropic, primarily designed to optimize software development workflows by facilitating tasks like reading code, making edits, running tests, and integrating with development environments. Here's a comprehensive look at how Claude Code is implemented and its architectural framework.

Key Features and Architecture

1. Modular Architecture:

2. Claude Code is structured around a well-defined architecture consisting of seven components:

- User Interface
- Agent Loop
- Permission System
- Tool Management
- State and Persistence
- Execution Environment

3. This architecture is structured across five layers, ensuring efficient interaction and task management within the coding environment [2].

4. Agentic Functionality:

5. At its core, Claude Code employs an agentic approach, enabling it to autonomously perform coding tasks. For instance, it can analyze existing codebases, suggest improvements, and execute specified tasks upon user commands. This includes automatic testing and contextual understanding of the overarching code structure [1] [3].

6. CLAUDE.md - Persistent Memory:

7. Claude Code utilizes a specific markdown file named CLAUDE.md, which serves as a project memory. This file stores vital instructions and project context that Claude uses to enhance its responses and suggestions across sessions, ensuring continuity in its interactions with developers [12].

8. Tool Integration and Command Execution:

9. The system seamlessly integrates tools for various functions such as file editing, command execution, and communication with external services. For example, it can run entire test suites after making code changes and automatically commit modifications to a version control system upon user approval [3].

10. AI Decision Logic and Infrastructure:

11. Interestingly, only a small fraction of Claude Code's codebase (around 1.6%) is dedicated to AI decision-making. This highlights the importance of infrastructure that supports AI functionality, including routing logic, context management, and error handling, making the entire system robust and dependable [2] [11].

12. Adaptive Agent Teams:

13. One of Claude Code's standout features is the ability to create and manage adaptive "agent teams." This allows multiple Claude agents to work concurrently on related tasks, enhancing efficiency and creativity in problem-solving [1] [5].

Development History and Usage

Claude Code was quickly adopted within Anthropic, with about 20% of the engineering team utilizing the tool shortly after its initial release. This rapid acceptance showcases the tool's effectiveness and utility in real-world coding environments [5].

Conclusions

Claude Code represents a significant evolution in software development tools, integrating AI capabilities with user-centered features to simplify coding tasks. By leveraging a blend of sophisticated architecture, persistent memory, and agentic functionalities, it stands as a critical resource for developers looking to enhance productivity and streamline their workflows.